

THE LOOK

Data Warehouse

ETL Pipeline & Data Modeling for E-Commerce Sales Analytics

Zafirah Aida Adista

CODA-RMT-015 | Graded Challenge 6

Google BigQuery · PySpark · PostgreSQL

Business Problem

The Look is an e-commerce company that requires a Data Warehouse to support comprehensive sales performance analysis. Transactional data scattered across various tables in Google BigQuery needs to be integrated into a single, structured, and centralized system.

01

Data Scattered

Sales data is scattered across multiple disparate tables

02

No Unified View

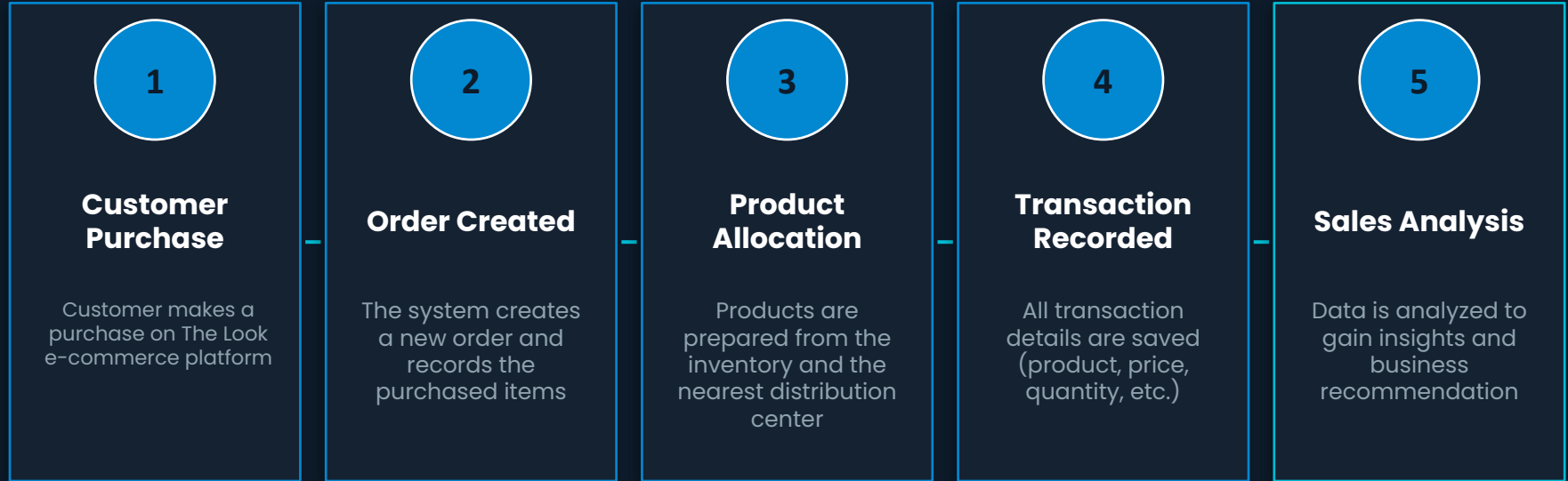
There is no centralized platform to analyze sales trends, product performance, and customer behavior

03

Analytics Need

Management requires fast and accurate analytical reports to support data-driven business decision-making

Business Process



Business Process Focus: This Data Warehouse focuses on the sales process, from order placement to transaction completion, covering the relationships between customers, products, locations, and transaction timestamps

Data Modeling

Design Background & Model Design

Design Background



Understanding Business Problems

The Look has various tables in BigQuery representing daily sales activities—from customer identification, products purchased, transaction timestamps, to the originating distribution center. This data cannot be analyzed directly as it is scattered and unstructured for reporting needs



Defining Business Process & Grain

The core business process is **sales per item per order**. From this, the grain is defined: one row in the Fact Table represents one item sold within an order



Why Star Schema?

- **Simplified Data Relationships:** Direct links between the Fact Table and Dimension Tables simplify the data joining process without complex structures.
- **Query Performance Optimization:** A concise structure allows for significantly faster and more responsive data retrieval.
- **Ease of Analyst Access:** The simple data structure enables analysts to quickly understand data flow for reporting requirements.

Fact Table

fact_sales : 1 row = 1 item sold in 1 order

Grain: order_item level

Measures: qty, sale_price, total_amount

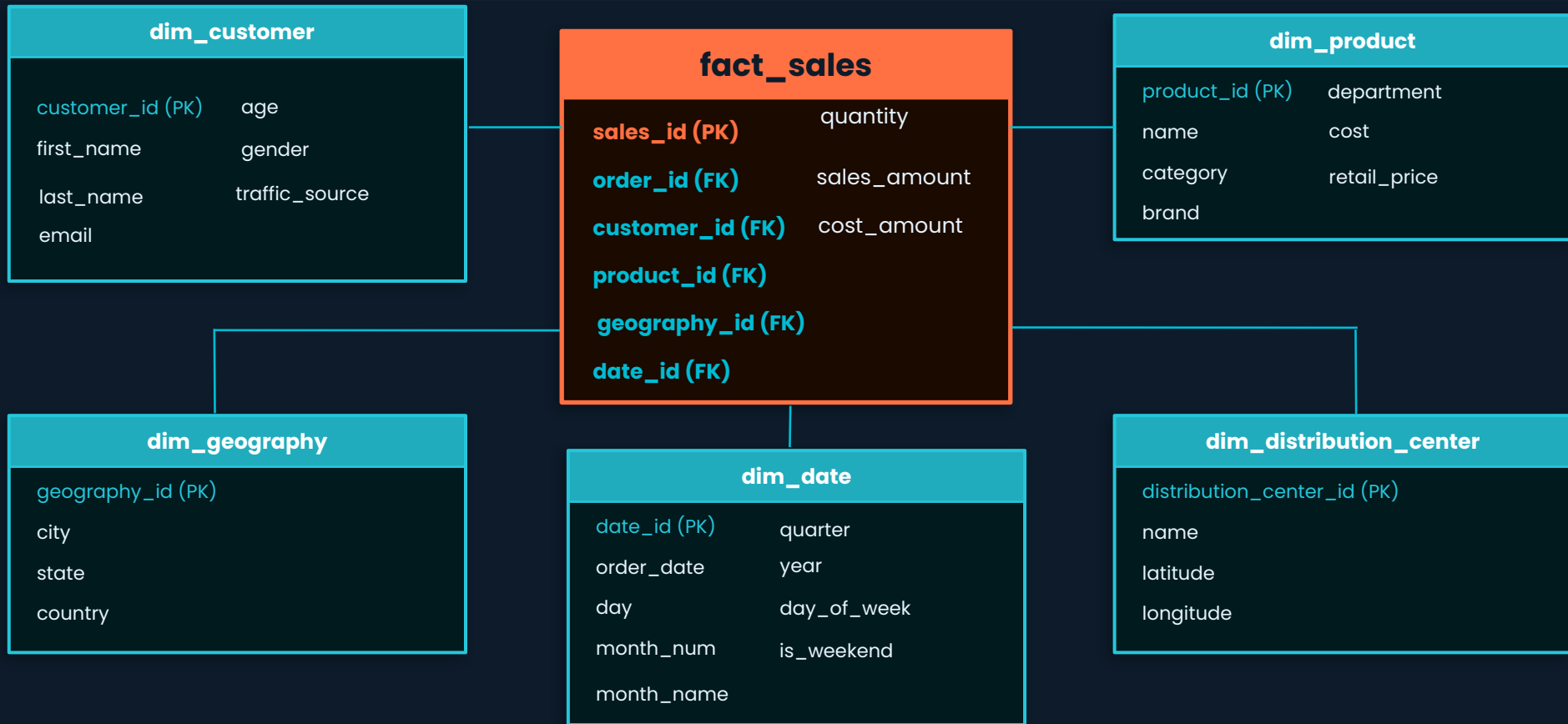
Dimension Tables (5 tabel)

dim_customer	← users	Who is the buyer?
dim_product	← products	Which product?
dim_geography	← users (city/country)	From where?
dim_date	← orders.created_at	When?
dim_distribution_center	← distribution_centers	Which distribution center?

Data Modeling - Skema

Star Schema Diagram

Star Schema

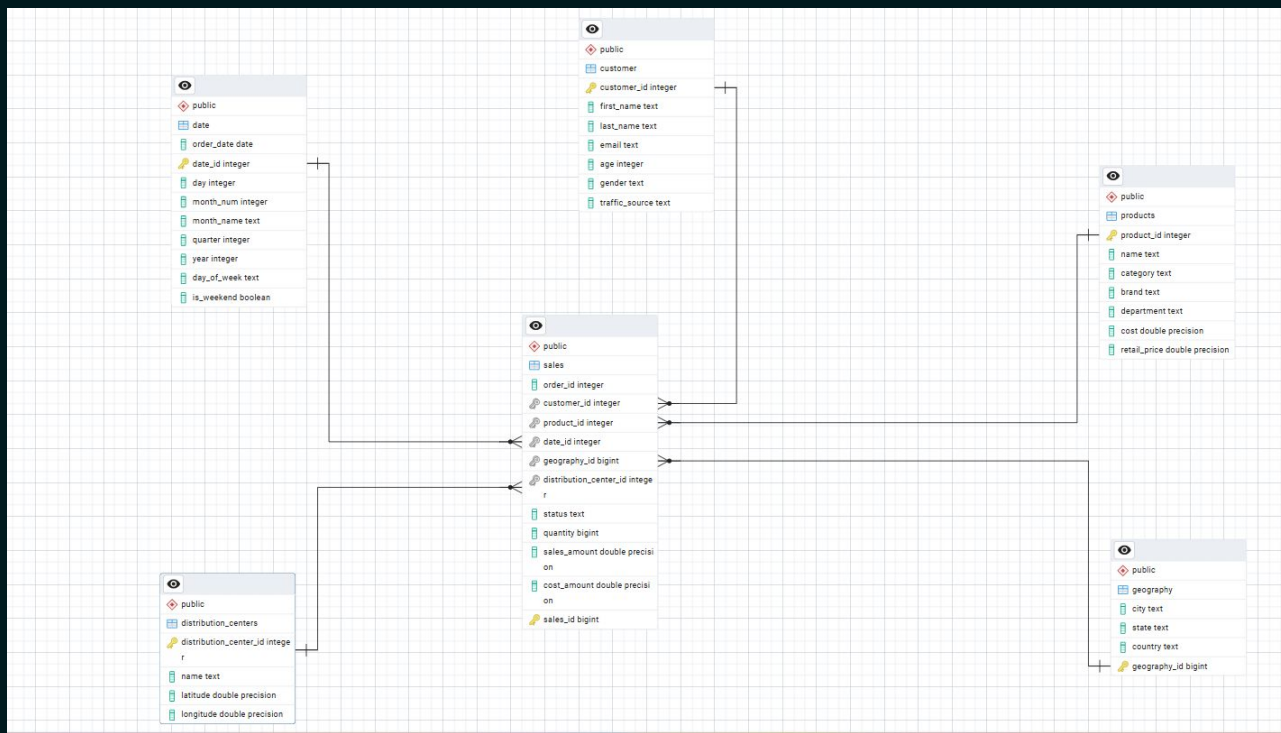


Data Modeling - Skema

Star Schema Diagram

Star Schema

ERD PostgreSQL



ETL Process – Overview

Extract · Transform · Load

1. EXTRACT



Google BigQuery → CSV → PySpark
DataFrame

- Fetch 6 tables from BigQuery
- Save as .csv files
- Load CSV into PySpark DataFrame

2. TRANSFORM



Cleaning + Modeling di PySpark

- Drop irrelevant columns
- Handle missing & null values
- Identify & remove duplicates
- Construct 5 Dimension + 1 Fact
Tables

3. LOAD



PySpark JDBC → PostgreSQL

- Establish connection to PostgreSQL
Database
- Utilize org.postgresql.Driver
- Upload Tabel Dim & Fact

ETL – Extract

Extracting 6 source tables from Google BigQuery

1. EXTRACT

Google BigQuery

File .csv

PySpark DataFrame

users

~100K

Customer Profile Data:

Id	Street_address	Latitude
First_name	City	Longitude
Last_name	State	User_geom
Email	Postal_code	Traffic_source
Age	Country	Created_at
Gender		

products

~29K

Product Catalog:

Id	Retail_price
Cost	Department
Category	Sku
Name	distribution_center_id
Brand	

orders

~125K

Order History & Transaction Status:

order_id	Returned_at
User_id	Shipped_at
Status	Delivered_at
Gender	num_of_item
Created_at	

order_items

~250K

Item Details per Order:

Id	Created_at
Order_id	Shipped_at
user_id	Delivered_at
Product_id	Returned_at
Inventory_item_id	sale_price
Status	

distribution_centers

10

Warehouse & Distribution Locations:

Id	
Name	
latitude	
Longitude	
distribution_center_geom	

inventory

~490K

Product-to-Distribution Center Relationship:

Product_id	
product_distribution_center_id	

ETL – Transform: Data Cleaning

Step 1 – Drop columns that are irrelevant to sales analysis

2. TRANSFORM

users

Dropped

street_address

postal_code

latitude

longitude

user_geom

- Detailed address and coordinate columns are not required. City, state, and country are sufficient for location analysis.
- Geom is a geometry format that is irrelevant for the Data Warehouse (DW).

orders

Dropped

gender

status

returned_at

shipped_at

delivered_at

num_of_item

- Gender is already available in the users table.
- Status and shipping/return dates are not the focus of this sales analysis.
- Num_of_item can be recalculated from the order_items table.

order_items

Dropped

inventory_item_id

shipped_at

delivered_at

returned_at

- Inventory_item_id is an internal stock ID not utilized in the DW.
- Shipped/delivered/returned dates are outside the scope of the sales analysis.

ETL – Transform: Data Cleaning

Step 1 – Drop columns that are irrelevant to sales analysis

2. TRANSFORM

products

Dropped

sku

SKU is an internal operational code that is not required for product performance analysis from a sales perspective

distribution_centers

Dropped

distribution_center_geom

Geom is a binary geometry format and is not required. Latitude and longitude coordinates are sufficient for warehouse location analysis.

ETL – Transform: Data Cleaning

Step 2 – Handling Missing Values, Null Strings & Duplicates

2. TRANSFORM

Missing Values: Found only in the products table

Kolom: **name** 2 rows null

dropna()

Two rows were removed as the number is very small and does not significantly impact the dataset.

Kolom: **brand** ~4K rows null

fillna("Unknown")

Empty brands likely indicate unbranded or generic products. The transaction data remains valid and should not be discarded; therefore, it is filled with 'Unknown' to allow for error-free aggregation

"Null" Strings : ditemukan di kolom city tabel users

Variasi ditemukan: "nul", "nules"

regex_replace() → diganti "Unknown"

These are not technical NULLs, so they bypass isNull() checks. Manual verification with string filters was required. The cause is likely input errors (typos of the word 'null' during data entry).

Duplikat : tidak ditemukan di semua tabel

Checks were performed across all tables, and no duplicate records were identified

ETL – Transform: Build Dimension Tables

Step 3 – Constructing 5 Dimension Tables based on the Star Schema design

2. TRANSFORM

dim_customer

← users (cleaned)

customer_id

first_name

last_name

email

age

gender

traffic_source

- Stores customer profiles
- ID column renamed from 'id' to 'customer_id' to clarify the Foreign Key (FK) in the Fact Table

dim_product

← products (cleaned)

product_id

name

category

brand

department

cost

retail_price

- Stores product details, including cost and retail_price for margin analysis
- ID renamed from 'id' to 'product_id'

dim_geography

← users (city, state, country)

geography_id

city

state

country

Separated from dim_customer to allow for independent location analysis

ETL – Transform: Build Dimension Tables

Step 3 – Constructing 5 Dimension Tables based on the Star Schema design

2. TRANSFORM

dim_date

← orders (created_at)

date_id

order_date

day

month_num

month_name

quarter

year

day_of_week

is_weekend

- Derived from the created_at column by extracting time components.
- date_id is created in YYYYMMDD integer format for efficient use as a Primary Key (PK).
- This table enables trend analysis based on day, month, quarter, year, or day of the week.

dim_distribution_center

← distribution_centers (cleaned)

distribution_center_id

name

latitude

longitude

- Stores distribution warehouse information.
- Latitude and longitude are retained for spatial analysis if required.
- ID column renamed from 'id' to 'distribution_center_id'.

ETL – Transform: Build Fact Table

Step 4 - Constructing fact_sales through multi-joins and aggregation

2. TRANSFORM

Join Strategy : 5 Tabel

JOIN orders

ON order_id

→ To retrieve user_id and transaction date (created_at)

JOIN products

ON product_id

→ To retrieve the product_id Foreign Key (FK).

JOIN users

ON user_id

→ To retrieve the customer_id FK and customer city.

JOIN inventory

ON product_id

→ To link products to their respective distribution_center

JOIN dim_geography

ON city + country

→ To retrieve the geography_id FK

groupBy Foreign Keys

order_id

Grain identifier

customer_id

FK dim_customer

product_id

FK dim_product

geography_id

FK dim_geography

date_id

FK dim_date

agg → Measures

quantity

count(item_id)

Number of items sold per transaction

sales_amount

sum(sale_price)

Total value per transaction.

cost_amount

Cost × quantity

Total cost of goods per transaction.

ETL – Load

Loading transformation results into PostgreSQL

3. LOAD

1

Establish connection
to PostgreSQL via
PySpark JDBC

2

Write Dataframe
to PostgreSQL

3

Upload Tabel
Dim & Fact

4

Verify data within
PostgreSQL

Urutan Load

1st

dim_customer

No dependencies

1st

dim_product

No dependencies

1st

dim_geography

No dependencies

1st

dim_date

No dependencies

1st

dim_distribution_center

No dependencies

2nd

fact_sales

Requires all Dimension tables

Thank You

Data Warehouse · The Look E-Commerce